Solution: Sum of Subarray Minimums

Problem Statement

Given an integer array `arr`, find the `sum` of the `minimum` elements of all possible `subarrays` of `arr`. Return the answer modulo $10^9 + 7$ as it may be very large.

Examples

Example 1:

- **Input**: [2, 3, 1]
- **Expected Output**: 10
- **Justification**: The subarrays are [2], [3], [1], [2, 3], [3, 1], and [2, 3, 1]. Their minimums are **2**, **3**, **1**, **2**, **1**, and **1** respectively. Summing these minimums yields **10**.

Example 2:

- **Input**: [4, 5, 2]
- **Expected Output**: 19
- **Justification**: The subarrays are [4], [5], [2], [4, 5], [5, 2], and [4, 5, 2]. Their minimums are **4**, **5**, **2**, **4**, **2**, and **2** respectively. Summing these minimums gives us **19**.

Example 3:

- **Input**: [7, 2, 8, 4]
- **Expected Output**: 35
- **Justification**: The subarrays are [7], [2], [8], [4], [7, 2], [2, 8], [8, 4], [7, 2, 8], [2, 8, 4], and [7, 2, 8, 4]. Their minimums are **7**, **2**, **8**, **4**, **2**, **2**, **4**, **2**, **2**, and **2** respectively. Summing these minimums totals **35**.

Solution

To solve this problem, we utilize a stack to efficiently track the minimum elements in all subarrays. The essence of this approach lies in its ability to maintain a stack of elements of the input array that are in ascending order. This method ensures that for any given element in the array, we can quickly determine the range of subarrays for which it is the minimum element. This approach is effective because it allows us to process each element of the array exactly once, thus ensuring optimal time complexity.

The rationale behind using a stack is that it enables us to keep track of elements in a way that, when we encounter a new element, we can efficiently find the nearest smaller element to its left and its right. This information is crucial for determining the contribution of each element to the final sum. By calculating the distance to the nearest smaller elements on both sides, we can determine how many subarrays each element is the minimum of, thereby allowing us to calculate its total contribution to the sum. This method is preferred for its efficiency and ability to handle the problem's requirements systematically.

Step-by-step Algorithm

1. **Initialization**:
   - Declare a stack that will store pairs of elements from the array and their frequency of being the minimum in subarrays.
   - Initialize two variables, `result` and `currentSum`, to store the final sum of subarray minimums and the sum of minimums for the current element being processed, respectively.
   - Define a modulo `mod` (e.g., `1e9 + 7`) to handle large sums and prevent integer overflow.
2. **Iterate Over the Array**:
   - For each element in the array, perform the following steps:
     - Initialize a variable `count` to 1. This variable tracks how many times the current element can be the minimum in its potential subarrays.
3. **Processing Elements in Stack**:
   - While the stack is not empty and the top element of the stack is greater than or equal to the current element:
     - Pop the top element from the stack. This element is no longer the minimum for any subarray extending beyond the current element.
     - Subtract the contribution of the popped element (its value multiplied by its count) from `currentSum`.
     - Add the count of the popped element to the `count` of the current element, as the current element will now be the minimum for those subarrays too.
4. **Update Stack and Current Sum**:
   - Push the current element and its `count` onto the stack. This action signifies that the current element is now being considered as a potential minimum for upcoming subarrays.
   - Update `currentSum` by adding the product of the current element's value and its `count`. This update reflects the new subarrays where the current element is the minimum.
   - Update the `result` by adding `currentSum` to it. `currentSum` represents the sum of minimums for subarrays ending at the current element.
5. **Finalizing the Result**:
   - After processing all elements in the array, the `result` variable will hold the sum of the minimums of all subarrays. Since we are using a modulo operation, ensure the final result is non-negative by adjusting with the modulo if necessary.
6. **Return the Result**:
   - Return the `result` as the final answer to the problem.

Algorithm Walkthrough

Let's take the input `[7, 2, 8, 4]`:

1. Initialize an empty stack and variables `result` and `currentSum` to 0.
   - Stack: `[]`
   - result: `0`
   - currentSum: `0`
2. Iterate through the input array `[7, 2, 8, 4]`:

   a. For the first element `7`:

   - Push `[7, 1]` onto the stack.
   - Update `currentSum` to `(0 + 7*1) % mod = 7`.
   - Update `result` to `(0 + 7) % mod = 7`.
   - Stack: `[[7, 1]]`
   - result: `7`
   - currentSum: `7`

   b. For the second element `2`:

   - Pop `[7, 1]` from the stack since `2 < 7`.
   - Increment `count` by 1 (from 1 to 2).
   - Update `currentSum` by subtracting `(7 * 1)` from it.
   - Push `[2, 2]` onto the stack.
   - Update `currentSum` to `(7 - 7*1 + 2*2) % mod = 4`.
   - Update `result` to `(7 + 4) % mod = 11`.
   - Stack: `[[2, 2]]`
   - result: `11`
   - currentSum: `4`

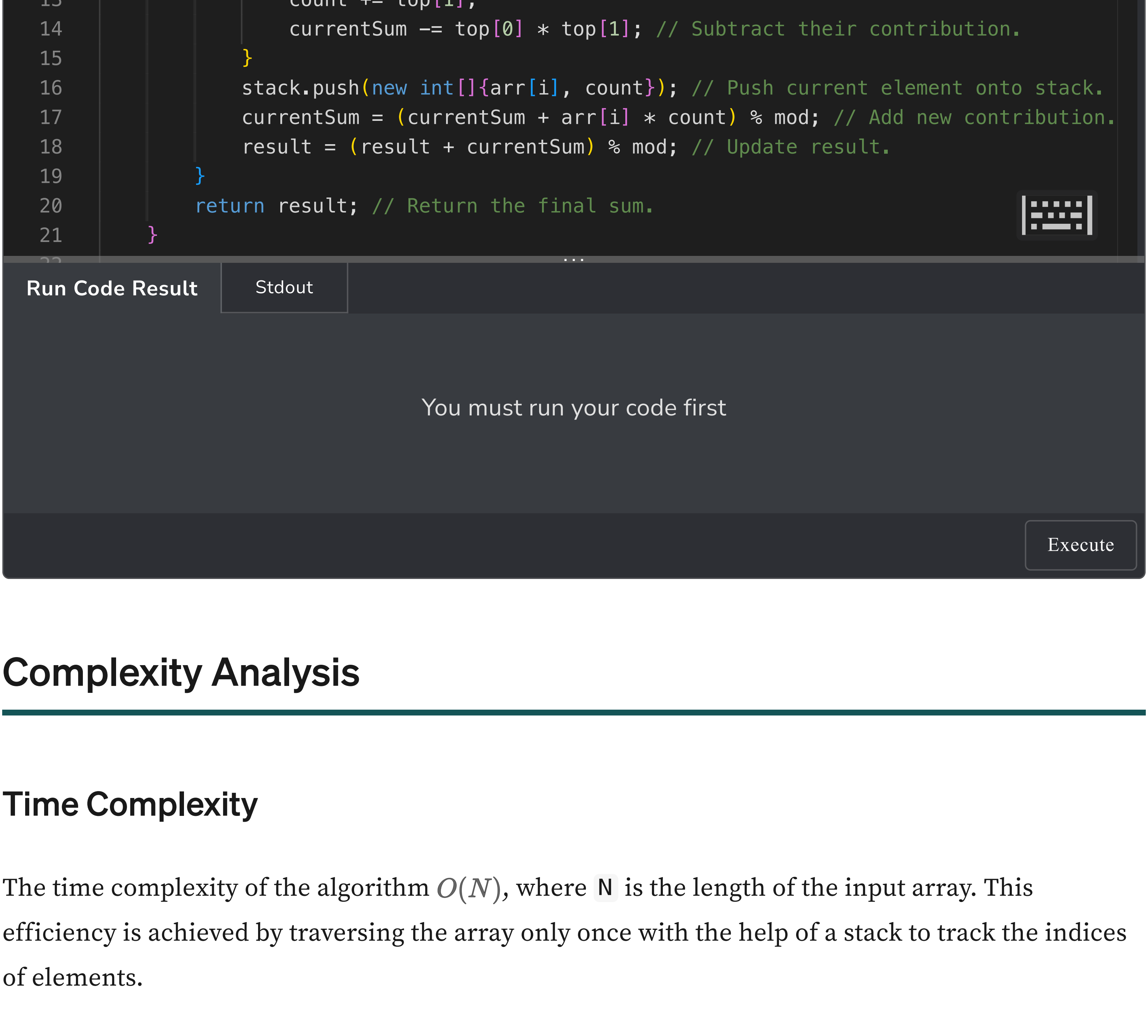
   c. For the third element `8`:

   - Push `[8, 1]` onto the stack.
   - Update `currentSum` to `(4 + 8*1) % mod = 12`.
   - Update `result` to `(11 + 12) % mod = 23`.
   - Stack: `[[2, 2], [8, 1]]`
   - result: `23`
   - currentSum: `14`

   d. For the fourth element `4`:

   - Pop `[8, 1]` from the stack since `4 < 8`.
   - Increment `count` by 1 (from 1 to 2).
   - Update `currentSum` by subtracting `(8 * 1)` from it.
   - Push `[4, 4]` onto the stack.
   - Update `currentSum` to `(12 - 8*1 + 4*2) % mod = 12`.
   - Update `result` to `(23 + 12) % mod = 35`.
   - Stack: `[[4, 4]]`
   - result: `35`
3. Return the final result, which is `35`.

Code



```java
import java.util.Stack;

public class Solution {
    public int sumSubarrayMins(int[] arr) {
        int mod = 1_000_000_007; // Define modulo for handling large sums.
        Stack<int[]> stack = new Stack<>(); // Stack to keep track of minimum elements.
        int result = 0, currentSum = 0; // Initialize result and currentSum.
        for (int i = 0; i < arr.length; i++) {
            int count = 1;
            // Pop elements from the stack if current element is smaller.
            while (!stack.isEmpty() && stack.peek()[0] >= arr[i]) {
                int[] top = stack.pop();
                count += top[1];
                currentSum -= top[0] * top[1]; // Subtract their contribution.
            }
            stack.push(new int[]{arr[i], count}); // Push current element onto stack.
            currentSum = (currentSum + arr[i] * count) % mod; // Add new contribution.
            result = (result + currentSum) % mod; // Update result.
        }
        return result; // Return the final sum.
    }
```

You must run your code first

Complexity Analysis

Time Complexity

The time complexity of the algorithm $O(N)$, where `N` is the length of the input array. This efficiency is achieved by traversing the array only once with the help of a stack to track the indices of elements.

For each element, operations within the loop (pushing to or popping from the stack) occur in constant time on average, since each element is pushed onto and popped from the stack exactly once.

Space Complexity

The space complexity is $O(N)$ due to the utilization of a stack that, in the worst case, might contain all `N` elements of the array if the array is sorted in ascending order. This stack is used to keep track of the elements' indices and their count of occurrences as minimums for the subarrays.

← Previous
Sum Of Subarray Minimums (Medium)
Next →
Rotate Image (Medium)
☐ Mark as Completed